
ShieldFi Protocol

Security Audit Report

Blockchain Technologies 2 — Final Project

Internal Team-Authored Audit

1. Executive Summary

ShieldFi is a parametric insurance protocol deployed on an Ethereum Layer 2 network. The protocol allows users to purchase price-trigger insurance backed by an ERC-4626 underwriting vault. This report covers a team-internal security review of all production Solidity contracts, tooling-assisted analysis with Slither, and documented reproduction and remediation of two representative vulnerability classes.

The audit found no Critical or High severity issues in production contracts. Three Low severity findings and one Informational item are documented below. All Low findings are mitigated by design constraints or accepted with documented rationale. The intentionally vulnerable lesson contracts (contracts/lessons/vulnerable/) are excluded from scope.

Audit Outcome Summary:

Severity	Count	Status
Critical	0	None found
High	0	None found
Medium	0	None found
Low	3	Documented and accepted / mitigated
Informational	1	Documented
Gas	1	Benchmarked and documented

2. Audit Scope

2.1 Commit Reference

Commit hash: [final submission commit hash — to be filled at submission]

Repository: [https://github.com/\[team-repo\]/Blockchain_Final](https://github.com/[team-repo]/Blockchain_Final)

Audit date: 2025

2.2 Files In Scope

File	Contract	Lines (approx.)
contracts/vault/InsuranceVault.sol	InsuranceVault	~120
contracts/vault/InsuranceVaultV2.sol	InsuranceVaultV2	~30
contracts/policy/PolicyManager.sol	PolicyManager	~180
contracts/claims/ClaimProcessor.sol	ClaimProcessor	~110
contracts/governance/ShieldProtocolGovernor.sol	ShieldProtocolGovernor	~60
contracts/token/ShieldGovToken.sol	ShieldGovToken	~40
contracts/token/PolicyNFT.sol	PolicyNFT	~50
contracts/treasury/ProtocolTreasury.sol	ProtocolTreasury	~50
contracts/amm/ShieldAMM.sol	ShieldAMM	~200
contracts/factory/PoolFactory.sol	PoolFactory	~40
contracts/factory/PolicyPool.sol	PolicyPool	~30
contracts/math/GasOptimizedMath.sol	GasOptimizedMath (library)	~30
contracts/interfaces/	AggregatorV3Interface, IPolicyManager, IClaimProcessor	~40

2.3 Files Out of Scope

- contracts/lessons/vulnerable/ — intentionally vulnerable contracts used for pedagogical demonstration only.
 - contracts/mocks/ — MockAggregatorV3, MockERC20 — test helpers not deployed to production.
 - test/ — test suite files.
 - lib/ — third-party library code (forge-std, OpenZeppelin).
 - script/ — deployment and verification scripts (reviewed informally but not formally audited).
-

3. Methodology

3.1 Tools Used

Tool	Version	Purpose
Slither	Latest (crytic/slither-action)	Automated static analysis — High and Medium gating in CI
Foundry / forge	Latest stable	Unit, fuzz, invariant, and fork testing
forge coverage	Built-in	Line coverage measurement (target: $\geq 90\%$)
solhint	Latest	Solidity linting (style and best practices)
forge fmt	Built-in	Formatting consistency
Manual review	Team	CEI pattern, access control, oracle checks, storage layout, upgrade safety

3.2 Manual Review Approach

Each auditor performed a structured manual review of every in-scope contract following these steps:

- Read all state variables and map trust assumptions to roles.
 - Trace every external call and verify CEI compliance or ReentrancyGuard presence.
 - Verify every privileged function is gated by AccessControl or Ownable.
 - Check ERC-20 interactions for use of SafeERC20.
 - Check Chainlink feed integration for staleness, bad answer, and incomplete round guards.
 - Verify UUPS upgrade authorization is restricted to UPGRADER_ROLE (Timelock).
 - Check storage layout for upgrade collision risks.
 - Verify no use of tx.origin, block.timestamp as randomness, or deprecated transfer/send.
-

4. Findings Table

ID	Title	Severity	Status
F-01	Governance parameter mismatch risk if token clock changes	Low	Documented / Monitor
F-02	ERC-4626 donation / inflation attack on empty vault	Low	Mitigated (OZ defaults + seeding guidance)
F-03	Intentional lesson contracts isolated from production scope	Informational	Accepted / Excluded
F-04	Chainlink feed assumptions (heartbeat, feed selection)	Low	Mitigated by staleness checks
G-01	Yul assembly path for premium calculation	Gas	Benchmarked

5. Detailed Findings

Finding F-01: Governance Parameter Mismatch Risk if Token Clock Changes

Field	Detail
Severity	Low
Location	contracts/governance/ShieldProtocolGovernor.sol, contracts/token/ShieldGovToken.sol
Status	Documented / Monitor

Description:

The ShieldProtocolGovernor uses block-number-based voting windows (initialVotingDelayBlocks, initialVotingPeriodBlocks). If the governance token's clock mode were changed to timestamp-based (via ERC-6372), the Governor's block-based parameters would become mismatched, potentially causing very long or very short voting periods on chains where block time differs significantly from expectations.

Impact:

Voting periods could become unexpectedly long or short following a token upgrade to timestamp-based clocking. This is a low-severity concern because the capstone uses block-number clocking consistently and no upgrade to timestamp mode is planned.

Proof of Concept:

If ShieldGovToken were upgraded to use block.timestamp as its clock (ERC-6372), and the Governor's votingPeriod were set to 50400 (intended as ~1 week at 12s/block), it would instead represent 50400 seconds (~14 hours), well below the intended period.

Recommendation:

If a timestamp-based clock is ever introduced, update all Governor parameters to use second-based values. Add a natspec warning to the constructor documenting the clock assumption.

Status: Accepted with documentation. No change required for capstone scope.

Finding F-02: ERC-4626 Donation / Inflation Attack on Empty Vault

Field	Detail
Severity	Low
Location	contracts/vault/InsuranceVault.sol (inherits OZ ERC4626Upgradeable)
Status	Mitigated

Description:

The classic ERC-4626 inflation attack: an attacker who is the first depositor can front-run a victim's deposit by donating assets directly to the vault contract (bypassing the deposit function), inflating the share price, and causing the victim to receive fewer shares than expected. This results in rounding losses for the victim.

Impact:

The OpenZeppelin ERC4626Upgradeable base contract applies virtual offset mitigation (adding a virtual 1 wei of assets and 1 share), which makes the attack economically infeasible in practice. Additionally, deployment guidance specifies that the deployer seeds the vault with a small initial deposit to prevent an empty-vault state.

Proof of Concept:

1. Vault is empty. Attacker calls deposit(1 wei) to become first LP. 2. Attacker transfers 1,000,000 tokens directly to the vault address (donation). 3. Victim calls deposit(1,000,000). Due to inflated share price, victim receives 0 shares (rounded down). 4. Attacker withdraws, stealing victim's deposit.

Recommendation:

The OZ virtual offset mitigates this in practice. Additionally, the deployment script should seed the vault with a minimum initial deposit immediately after initialization, before any user can interact with it.

Status: Mitigated by OZ virtual offset + seeding guidance in deployment script.

Finding F-03: Intentional Lesson Contracts (Informational)

Field	Detail
Severity	Informational
Location	contracts/lessons/vulnerable/VulnerableEthReentrancyLesson.sol, contracts/lessons/vulnerable/VulnerableMintAccessLesson.sol
Status	Accepted / Excluded from production scope

Description:

The repository contains two intentionally vulnerable contracts for pedagogical purposes. VulnerableEthReentrancyLesson.sol demonstrates an ETH reentrancy vulnerability by calling an external address before updating state. VulnerableMintAccessLesson.sol demonstrates missing access control on a mint function.

Impact:

None on production. These contracts are never referenced by production code and are excluded from Slither CI gating via filter_paths configuration.

Status: Accepted. Lesson contracts are intentional and clearly labeled.

Finding F-04: Chainlink Feed Heartbeat and Feed Selection Assumptions

Field	Detail
Severity	Low
Location	contracts/claims/ClaimProcessor.sol
Status	Mitigated

Description:

The ClaimProcessor accepts a heartbeatSeconds parameter at construction that is used for staleness validation. If the wrong heartbeat is configured for a given Chainlink feed (e.g., a 1-hour heartbeat feed configured with a 24-hour staleness tolerance), stale prices could be

accepted. Additionally, the contract does not validate that the feed address is not address(0) beyond the zero-check in the constructor.

Impact:

Stale prices accepted due to misconfiguration would allow incorrect claim settlements. However, the existing guards (answer > 0, answeredInRound >= roundId, future timestamp check) catch the most dangerous oracle failure modes regardless of heartbeat configuration.

Recommendation:

Document the required heartbeat value for each supported feed in the deployment script and in operator documentation. Consider emitting an event when the feed address or heartbeat is updated, to allow off-chain monitoring.

Status: Mitigated by existing staleness, bad answer, and incomplete round checks. Deployment documentation notes the correct heartbeat per feed.

6. Reproduced and Fixed Vulnerability Case Studies

6.1 Case Study 1 — ETH Reentrancy

6.1.1 Vulnerable Contract: `VulnerableEthReentrancyLesson.sol`

The vulnerable contract calls an external address before updating its internal balance mapping, enabling a malicious contract to re-enter `withdraw()` and drain funds.

Vulnerable pattern:

```
function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0);
    (bool ok,) = msg.sender.call{value: amount}(""); // External call BEFORE state
update
    require(ok);
    balances[msg.sender] = 0; // State update AFTER — too late
}
```

The attacker contract's `receive()` function calls `withdraw()` again before `balances[msg.sender]` is zeroed, draining the contract.

Proof of Concept test — `test/Lessons.t.sol`:

```
function test_reentrancyDrainsVulnerable() public {
    // Fund the vulnerable contract
    vm.deal(address(vuln), 1 ether);
    // Attacker deposits then re-enters during withdraw
    attacker.attack{value: 0.1 ether}();
    assertEq(address(vuln).balance, 0); // Drained
}
```

6.1.2 Fixed Contract: `FixedEthReentrancyLesson.sol`

The fix applies Checks-Effects-Interactions pattern and adds `ReentrancyGuard`:

```
function withdraw() external nonReentrant {
    uint256 amount = balances[msg.sender];
    require(amount > 0);
    balances[msg.sender] = 0; // State update BEFORE external call (CEI)
    (bool ok,) = msg.sender.call{value: amount}("");
    require(ok);
}
```

After fix — `test/Lessons.t.sol` confirms the attack reverts:

```
function test_reentrancyFixedReverts() public {
    vm.expectRevert();
    attacker.attackFixed{value: 0.1 ether}();
}
```

6.2 Case Study 2 — Missing Access Control on Mint

6.2.1 Vulnerable Contract: `VulnerableMintAccessLesson.sol`

A mint function lacks any access control, allowing any external caller to mint tokens to any address:

```
function mint(address to, uint256 amount) external {
    // No access control – any caller can mint
    _mint(to, amount);
}
```

Proof of Concept test:

```
function test_anyoneCanMint() public {
    address attacker = makeAddr("attacker");
    vm.prank(attacker);
    token.mint(attacker, 1_000_000e18); // Succeeds – no control
    assertEq(token.balanceOf(attacker), 1_000_000e18);
}
```

6.2.2 Fixed Contract: FixedMintAccessLesson.sol

The fix introduces AccessControl with a MINTER_ROLE, and the mint function checks for this role:

```
bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");

function mint(address to, uint256 amount) external onlyRole(MINTER_ROLE) {
    _mint(to, amount);
}
```

After fix — test confirms unauthorized mint reverts:

```
function test_unauthorizedMintReverts() public {
    address attacker = makeAddr("attacker");
    vm.prank(attacker);
    vm.expectRevert(); // AccessControl: missing role
    token.mint(attacker, 1_000_000e18);
}
```

7. CEI Pattern and ReentrancyGuard Coverage

Every externally callable function was reviewed for CEI compliance and ReentrancyGuard usage. The table below summarizes coverage for all state-changing functions in production contracts.

Contract	Function	CEI Compliant	ReentrancyGuard	Notes
InsuranceVault	deposit / withdraw / redeem / payClaim	Yes	Yes (nonReentrant)	Inherited from OZ ERC4626; payClaim updates totalPayouts before transfer
PolicyManager	buyPolicy	Yes	Yes (nonReentrant)	Premium split to treasury and vault before emitting event
ClaimProcessor	processClaim	Yes	Indirect (via vault)	Marks claimed before calling vault.payClaim
ShieldAMM	addLiquidity / removeLiquidity / swap	Yes	Yes (nonReentrant)	Reserve state updated before token transfers
ProtocolTreasury	withdraw	Yes	N/A (Timelock-only)	Timelock callers cannot reenter; SafeERC20 used
ShieldGovToken	mint / burn	Yes	N/A	No external calls
PolicyNFT	mint / burn	Yes	N/A	No external calls

8. Access Control Audit

All privileged functions use OpenZeppelin AccessControl or Ownable. The table below lists every privileged function and its required role.

Contract	Function	Required Role / Guard
InsuranceVault	pause() / unpause()	PAUSER_ROLE
InsuranceVault	payClaim()	CLAIM_PAYER_ROLE
InsuranceVault	upgradeTo() (UUPS)	UPGRADER_ROLE
PolicyManager	pause() / unpause()	PAUSER_ROLE
PolicyManager	setTreasuryFeeBps() / setAnnualPremiumBps()	DEFAULT_ADMIN_ROLE (Timelock)
PolicyManager	upgradeTo() (UUPS)	UPGRADER_ROLE
PolicyNFT	mint()	MINTER_ROLE
ProtocolTreasury	withdraw()	msg.sender == timelock (hardcoded check)
ShieldProtocolGovernor	setVotingDelay / setVotingPeriod / setProposalThreshold	Governance (self-executed via Timelock)
TimelockController	schedule / cancel	PROPOSER_ROLE (Governor)
TimelockController	execute	EXECUTOR_ROLE (address(0) — permissionless after delay)

No unguarded admin functions were found in production contracts. The use of tx.origin for authorization was not found. No use of block.timestamp as a source of randomness was found. ETH transfers use call{value:} with success checks; transfer/send were not used.

9. Centralization Analysis

The following analysis documents who holds power in the protocol and the consequences of compromise at each level.

Role / Key	Current Holder	Power	Compromise Consequence	Mitigation
DEFAULT_ADMIN_ROLE	Deployer, then transferred to Timelock	Grant / revoke any role	Full role takeover — but only after 2-day Timelock delay	Timelock delay provides community response window
UPGRADER_ROLE	Timelock	Replace contract logic via UUPS	Malicious implementation can drain funds — requires DAO vote + 2-day delay	Governance vote + Timelock delay required
PAUSER_ROLE	Deployer multisig	Halt all deposits, withdrawals, purchases	DOS — underwriters cannot exit	Timelock can be used to grant emergency un-pause authority
CLAIM_PAYER_ROLE	ClaimProcessor (EOA at deploy, then contract)	Trigger vault payouts	Unauthorized payouts — per-claim checks prevent excess beyond policy coverage	Claim amount bounded by <code>policy.coverageAmount</code>
Chainlink Feed	Chainlink network	Determine claim trigger prices	Stale / manipulated prices could trigger or block claims	Staleness check, bad answer check, <code>answeredInRound</code> check
TimelockController	Governor + 2-day delay	Execute any governance action	Proposal spam or flash-loan vote attack	Quorum 4%, threshold 1%, ERC20Votes checkpoints prevent flash loans

10. Governance Attack Analysis

10.1 Flash-Loan Governance Attack

Threat: An attacker borrows a large quantity of governance tokens via a flash loan, votes with the borrowed tokens in a single transaction, and repays the loan. This could allow a temporary large holder to pass a malicious proposal.

Mitigation: The ShieldGovToken uses ERC20Votes checkpoints. Voting power is measured at the block of the proposal snapshot, not the current block. A flash loan executed and repaid within one block has zero checkpointed voting power at the snapshot. The attack is infeasible.

10.2 Whale Vote Attack

Threat: A large token holder accumulates enough tokens to unilaterally meet quorum and proposal threshold, passing a malicious proposal.

Mitigation: Quorum is set at 4% of total supply, and proposal threshold at 1%. These do not prevent a whale attack if a single holder controls >4%. For production, a veto multisig or a guardian role (with limited powers) would be recommended. For capstone scope, this is an accepted risk with documentation.

10.3 Proposal Spam Attack

Threat: An attacker submits many proposals simultaneously to exhaust the governance queue, blocking legitimate proposals.

Mitigation: The proposalThreshold (1% of supply) requires meaningful token holdings to propose, raising the cost of spam. The Timelock can be paused by governance in extreme cases.

10.4 Timelock Bypass

Threat: An attacker with EXECUTOR_ROLE could execute operations before the delay expires.

Mitigation: The TimelockController enforces minDelay on-chain. EXECUTOR_ROLE is granted to address(0) (permissionless execution), but only after the delay has elapsed. Cancellation within the delay window is possible by the CANCELLER_ROLE (held by the multisig). No bypass of the minDelay is possible by any role.

11. Oracle Attack Analysis

11.1 Price Manipulation

Threat: An attacker manipulates the Chainlink feed price (e.g., via a large trade on the spot market) to trigger a false claim payout.

Mitigation: Chainlink aggregator prices are volume-weighted medians across multiple data sources. Spot market manipulation does not directly control the aggregator output. For production, using TWAP-based oracles would add further protection.

11.2 Stale Price Attack

Threat: If the Chainlink feed stops updating (feed pause, network congestion), the ClaimProcessor could accept stale prices.

Mitigation: ClaimProcessor validates: `require(updatedAt >= block.timestamp - heartbeatSeconds)` and rejects prices where `updatedAt > block.timestamp`. If the feed is stale, `processClaim` reverts with a staleness error, freezing claims until the feed recovers or governance updates the feed address.

11.3 Feed Address Depeg / Swap Attack

Threat: If the feed address is changed via governance to a malicious or misconfigured aggregator, false prices could be submitted.

Mitigation: Feed address changes require a governance proposal, DAO vote, and 2-day Timelock delay. This provides ample time for the community to detect and cancel a malicious proposal.

Oracle Check	Implementation	Protects Against
Staleness check	<code>require(updatedAt >= block.timestamp - heartbeatSeconds)</code>	Stale / halted feed
Future timestamp check	<code>require(updatedAt <= block.timestamp)</code>	Replay / time-manipulation
Bad answer check	<code>require(answer > 0)</code>	Feed returning 0 or negative
Incomplete round check	<code>require(answeredInRound >= roundId)</code>	Incomplete aggregation rounds

12. Gas Optimization Report

12.1 Yul Assembly vs Solidity — Premium Calculation

The GasOptimizedMath library implements premium calculation using both a Solidity path (OpenZeppelin Math.mulDiv) and an inline Yul assembly path. The Yul path is benchmarked against the Solidity equivalent:

Function	Approach	Gas (typical)	Difference
premiumMulDivSolidity	OZ Math.mulDiv (256-bit overflow safe)	~350 gas	Baseline
premiumMulDivYul	Inline assembly mul/div (bounded inputs)	~80 gas	~270 gas cheaper (~77% reduction)

Note: The Yul path is only safe for inputs where $\text{coverage} * \text{rateBps} * \text{duration}$ fits within 256 bits. The test suite enforces this constraint via input bounds in fuzz tests. For production use with unbounded inputs, Math.mulDiv should be preferred for safety.

12.2 Packed Storage

The Policy struct packs purchasedAt (uint48), expiresAt (uint48), triggerAbove (bool), claimed (bool), and active (bool) together, reducing storage slots used per policy by approximately 2 slots compared to unpacked uint256 fields.

12.3 uint128 for totalPayouts

InsuranceVault uses uint128 for totalPayouts instead of uint256, saving one storage slot when combined with adjacent packed data.

13. Slither Output (Appendix)

Slither static analysis is run in CI via `crytic/slither-action` with `fail-on: high`. The `filter_paths` configuration excludes mocks, lesson contracts, tests, scripts, and library dependencies. The following summarizes findings from the most recent Slither run on production contracts:

Detector	Severity	Finding	Disposition
reentrancy-no-eth	High	0 findings in production contracts	Passes CI gate
uninitialized-state	High	0 findings	Passes CI gate
suicidal	High	0 findings	Passes CI gate
shadowing-state	Medium	0 findings in production	Passes CI gate
events-access	Low	Reported on some admin setters — informational	Accepted: events emitted in all state-changing paths
dead-code	Informational	None in production scope	Clean
assembly	Informational	GasOptimizedMath.premiumMulDivYul	Intentional, benchmarked

Full Slither output is available in `slither_output.txt` in the repository root. CI configuration can be found in `.github/workflows/shieldfi-ci.yml`.

14. Conclusion

The ShieldFi protocol demonstrates sound security engineering practices for a capstone decentralized insurance protocol. Key strengths identified during this audit include:

- Consistent application of CEI pattern and ReentrancyGuard across all externally callable, fund-handling functions.
- Comprehensive role-based access control with no unguarded privileged functions found.
- Chainlink integration with four-layer validation (staleness, future timestamp, bad answer, incomplete round).
- UUPS upgradeability gated to a Timelock-controlled UPGRADER_ROLE, preventing unauthorized upgrades.
- Two complete vulnerability reproductions (reentrancy, access control) with before/after tests demonstrating both the vulnerability and the remediation.
- Zero Slither High and Medium findings in production contract scope at CI gate.

Low severity findings are fully documented with mitigations. No critical or high findings were identified. The protocol meets the security requirements for the Blockchain Technologies 2 final project capstone.

Recommended next steps before any production mainnet deployment:

- Commission a professional third-party security audit from a recognized Solidity security firm.
 - Implement a Pausable guardian multisig with a defined emergency response playbook.
 - Conduct a bug bounty program prior to mainnet launch.
 - Add a TWAP-based secondary oracle for claim settlement as defense-in-depth against price manipulation.
-